

Student System in Python

Name: Gaurav Nath

Roll no: CSB24073

Subject: Advance Programming

Semester: Spring 26

Assignment Question

Design a student system in Python with:

- Address class (street, city, zipCode)
- Student class with name, age, Address, and course list
- Store age as a protected attribute and control it using @property
- Methods: add_course() and display()

Extend it with:

- ScholarshipStudent (add scholarshipAmount and override display())

Your implementation should clearly show:

- Composition (Student HAS-A Address)
- Proper data validation using @property (age must be valid)
- Inheritance and overriding (use super() in display)
- Understanding of mutable behavior (course list updates persist)

Python Implementation

```
# ----- Address Class -----  
class Address:  
    def __init__(self, street, city, zipCode):  
        self.street = street  
        self.city = city  
        self.zipCode = zipCode  
  
    def display(self):  
        return f"{self.street}, {self.city} - {self.zipCode}"
```

```

# ----- Student Class -----
class Student:
    def __init__(self, name, age, address, courses=None):
        self.name = name
        self._age = None
        self.age = age
        self.address = address
        self.courses = courses if courses is not None else []

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if not isinstance(value, int) or value <= 0 or value > 120:
            raise ValueError("Age must be a valid positive integer (1-120)")
        self._age = value

    def add_course(self, course):
        self.courses.append(course)

    def display(self):
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")
        print(f"Address: {self.address.display()}")
        print(f"Courses: {'', ' '.join(self.courses) if self.courses else 'None'}")

# ----- ScholarshipStudent Class -----
class ScholarshipStudent(Student):
    def __init__(self, name, age, address, scholarshipAmount, courses=None):
        super().__init__(name, age, address, courses)
        self.scholarshipAmount = scholarshipAmount

    def display(self):
        super().display()
        print(f"Scholarship Amount: ₹{self.scholarshipAmount}")

# ----- Testing -----
addr = Address("MG Road", "Tezpur", "784001")

s1 = Student("Gaurav", 20, addr)
s1.add_course("Math")

```

```

s1.add_course("Python")
s1.display()

s2 = ScholarshipStudent("Anwasha", 21, addr, 50000)
s2.add_course("AI")
s2.display()

```

INPUT Screenshots

```

assignment 10 > system.py > ...
1  # ----- Address Class -----
2  class Address:
3      def __init__(self, street, city, zipCode):
4          self.street = street
5          self.city = city
6          self.zipCode = zipCode
7
8      def display(self):
9          return f"{self.street}, {self.city} - {self.zipCode}"
10
11
12  # ----- Student Class -----
13  class Student:
14      def __init__(self, name, age, address, courses=None):
15          self.name = name
16          self._age = None # protected attribute
17          self.age = age # setter will validate
18          self.address = address # HAS-A relationship (Composition)
19          self.courses = courses if courses is not None else []
20
21      # ----- Property for age (Validation) -----
22      @property
23      def age(self):
24          return self._age
25
26      @age.setter
27      def age(self, value):
28          if not isinstance(value, int) or value <= 0 or value > 120:
29              raise ValueError("Age must be a valid positive integer (1-120)")
30          self._age = value
31
32      # ----- Add Course -----
33      def add_course(self, course):
34          self.courses.append(course)
35
36      # ----- Display -----
37      def display(self):
38          print(f"Name: {self.name}")
39          print(f"Age: {self.age}")
40          print(f"Address: {self.address.display()}")
41          print(f"Courses: {' '.join(self.courses) if self.courses else 'None'}")
42

```

```

# ----- ScholarshipStudent Class -----
class ScholarshipStudent(Student):
    def __init__(self, name, age, address, scholarshipAmount, courses=None):
        super().__init__(name, age, address, courses)
        self.scholarshipAmount = scholarshipAmount

    # ----- Override display -----
    def display(self):
        super().display()
        print(f"Scholarship Amount: ₹{self.scholarshipAmount}")

# ----- Testing -----
if __name__ == "__main__":
    # Create Address object
    addr = Address("MG Road", "Tezpur", "784001")

    # Create Student object
    s1 = Student("Gaurav", 20, addr)
    s1.add_course("Math")
    s1.add_course("Python")

    print("\n--- Student Details ---")
    s1.display()

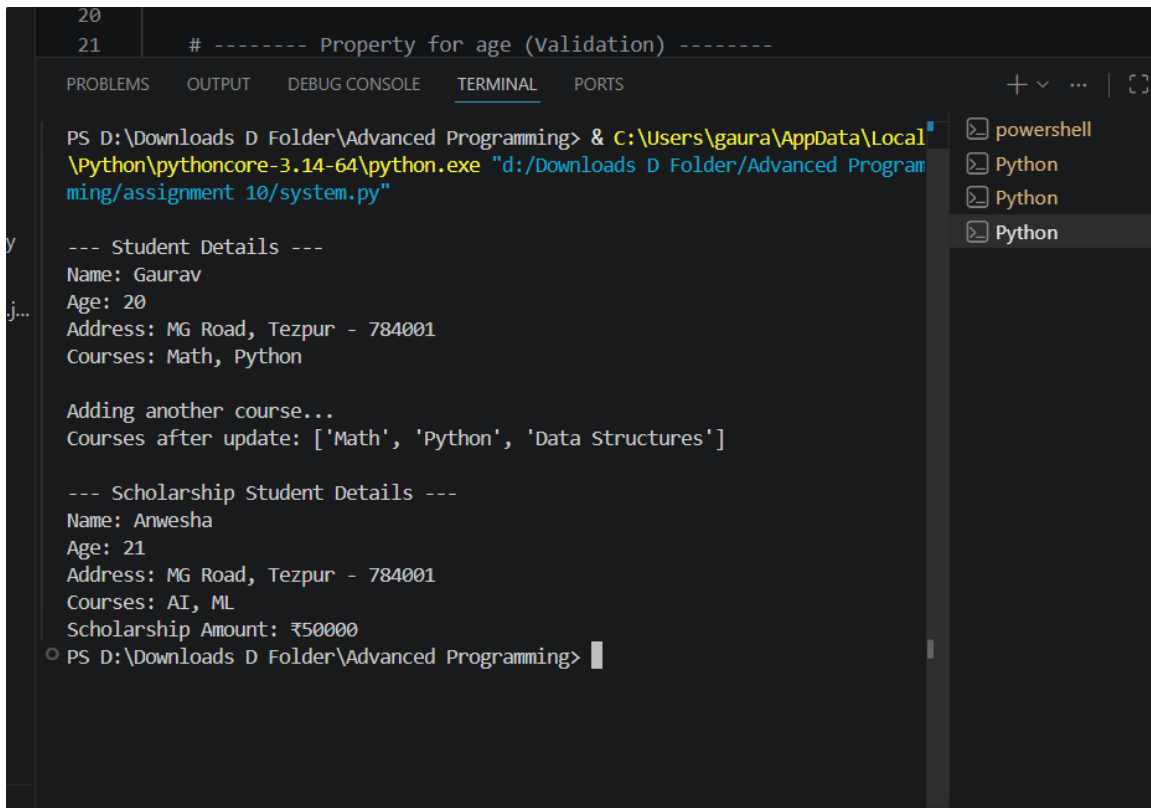
    # Mutable behavior check
    print("\nAdding another course...")
    s1.add_course("Data Structures")
    print("Courses after update:", s1.courses)

    # Create ScholarshipStudent object
    s2 = ScholarshipStudent("Anwesha", 21, addr, 50000)
    s2.add_course("AI")
    s2.add_course("ML")

    print("\n--- Scholarship Student Details ---")
    s2.display()

```

Output Screenshots



```
20
21 # ----- Property for age (Validation) -----

PS D:\Downloads D Folder\Advanced Programming> & C:\Users\gaura\AppData\Local\
Python\pythoncore-3.14-64\python.exe "d:/Downloads D Folder/Advanced Program
ming/assignment 10/system.py"

--- Student Details ---
Name: Gaurav
Age: 20
Address: MG Road, Tezpur - 784001
Courses: Math, Python

Adding another course...
Courses after update: ['Math', 'Python', 'Data Structures']

--- Scholarship Student Details ---
Name: Anwesha
Age: 21
Address: MG Road, Tezpur - 784001
Courses: AI, ML
Scholarship Amount: ₹50000
PS D:\Downloads D Folder\Advanced Programming>
```